

1) Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given

Employee and Department relations to retrieve required records.

```
mysql> SELECT * FROM Employee
-> WHERE Salary > 50000;
+----+-----+-----+-----+
| EID | EName | DeptID | Salary |
+----+-----+-----+-----+
| 101 | Arun  | 1      | 60000  |
| 103 | Kumar | 1      | 70000  |
| 105 | Siva  | 2      | 55000  |
+----+-----+-----+-----+
3 rows in set (0.02 sec)

mysql> SELECT EName
-> FROM Employee;
+-----+
| EName |
+-----+
| Arun  |
| Bala  |
| Kumar |
| Rahul |
| Siva  |
+-----+
5 rows in set (0.00 sec)

mysql> SELECT EName
-> FROM Employee
-> WHERE DeptID=1
-> UNION
-> SELECT EName
-> FROM Employee
-> WHERE DeptID=2;
+-----+
| EName |
+-----+
| Arun  |
| Kumar |
| Bala  |
| Siva  |
+-----+
```

```
mysql> SELECT *
-> FROM Employee
-> WHERE DeptID<>1;
+----+-----+-----+-----+
| EID | EName | DeptID | Salary |
+----+-----+-----+-----+
| 102 | Bala  | 2      | 45000  |
| 105 | Siva  | 2      | 55000  |
| 104 | Rahul | 3      | 40000  |
+----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT *
-> FROM Employee
-> CROSS JOIN Department;
+----+-----+-----+-----+-----+-----+
| EID | EName | DeptID | Salary | DeptID | DeptName |
+----+-----+-----+-----+-----+-----+
| 101 | Arun  | 1      | 60000  | 3      | Sales    |
| 101 | Arun  | 1      | 60000  | 2      | HR       |
| 101 | Arun  | 1      | 60000  | 1      | IT       |
| 102 | Bala  | 2      | 45000  | 3      | Sales    |
| 102 | Bala  | 2      | 45000  | 2      | HR       |
| 102 | Bala  | 2      | 45000  | 1      | IT       |
| 103 | Kumar | 1      | 70000  | 3      | Sales    |
| 103 | Kumar | 1      | 70000  | 2      | HR       |
| 103 | Kumar | 1      | 70000  | 1      | IT       |
| 104 | Rahul | 3      | 40000  | 3      | Sales    |
| 104 | Rahul | 3      | 40000  | 2      | HR       |
| 104 | Rahul | 3      | 40000  | 1      | IT       |
| 105 | Siva  | 2      | 55000  | 3      | Sales    |
| 105 | Siva  | 2      | 55000  | 2      | HR       |
| 105 | Siva  | 2      | 55000  | 1      | IT       |
+----+-----+-----+-----+-----+-----+
15 rows in set (0.01 sec)
```

```

15 rows in set (0.01 sec)

mysql> SELECT *
      -> FROM Employee
      -> INNER JOIN Department
      -> ON Employee.DeptID=Department.DeptID;
+-----+-----+-----+-----+-----+-----+
| EID | EName | DeptID | Salary | DeptID | DeptName |
+-----+-----+-----+-----+-----+-----+
| 101 | Arun  | 1      | 60000  | 1      | IT       |
| 103 | Kumar | 1      | 70000  | 1      | IT       |
| 102 | Bala  | 2      | 45000  | 2      | HR       |
| 105 | Siva  | 2      | 55000  | 2      | HR       |
| 104 | Rahul | 3      | 40000  | 3      | Sales    |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

```

- 2) Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

```

mysql> DESC STUDENT;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| SID   | int           | NO   | PRI | NULL    |       |
| Name  | varchar(30)   | YES  |     | NULL    |       |
| DOB   | date          | YES  |     | NULL    |       |
| DeptID | int           | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.06 sec)

mysql> DESC DEPARTMENT;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| DeptID     | int           | NO   | PRI | NULL    |       |
| DeptName   | varchar(30)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

3) Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

```
mysql> SELECT COUNT(*) FROM Sales;
+-----+
| COUNT(*) |
+-----+
| 5 |
+-----+
1 row in set (0.06 sec)

mysql> SELECT SUM(Amount) FROM Sales;
+-----+
| SUM(Amount) |
+-----+
| 112700 |
+-----+
1 row in set (0.05 sec)

mysql> SELECT AVG(Amount) FROM Sales;
+-----+
| AVG(Amount) |
+-----+
| 22540.0000 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT MAX(Amount) FROM Sales;
+-----+
| MAX(Amount) |
+-----+
| 60000 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> SELECT MIN(Amount) FROM Sales;
+-----+
| MIN(Amount) |
+-----+
| 500 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT Product, SUM(Amount)
-> FROM Sales
-> GROUP BY Product;
+-----+-----+
| Product | SUM(Amount) |
+-----+-----+
| Laptop | 110000 |
| Mouse | 1200 |
| Keyboard | 1500 |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT Product, SUM(Amount)
-> FROM Sales
-> GROUP BY Product
-> HAVING SUM(Amount)>1000;
+-----+-----+
| Product | SUM(Amount) |
+-----+-----+
| Laptop | 110000 |
| Mouse | 1200 |
| Keyboard | 1500 |
+-----+-----+
3 rows in set (0.00 sec)
```

- 4) Write a correlated sub-query to find employees earning more than the average salary of their own department.

```
mysql> SELECT *
-> FROM Employee e
-> WHERE Salary >
-> (
-> SELECT AVG(Salary)
-> FROM Employee
-> WHERE DeptID=e.DeptID
-> );
```

EID	EName	DeptID	Salary
103	Kumar	1	70000
105	Siva	2	55000

2 rows in set (0.06 sec)

- 5) Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.

```
mysql> SELECT *
-> FROM Employee
-> INNER JOIN Project
-> ON Employee.EID=Project.EID;
```

EID	EName	DeptID	Salary	PID	ProjectName	EID
101	Arun	1	60000	1	Website	101
103	Kumar	1	70000	2	App	103
105	Siva	2	55000	3	Software	105

3 rows in set (0.00 sec)

```
mysql> SELECT *
-> FROM Employee
-> LEFT JOIN Project
-> ON Employee.EID=Project.EID;
```

EID	EName	DeptID	Salary	PID	ProjectName	EID
101	Arun	1	60000	1	Website	101
102	Bala	2	45000	NULL	NULL	NULL
103	Kumar	1	70000	2	App	103
104	Rahul	3	40000	NULL	NULL	NULL
105	Siva	2	55000	3	Software	105

5 rows in set (0.00 sec)

```
mysql> SELECT *
-> FROM Employee
-> RIGHT JOIN Project
-> ON Employee.EID=Project.EID;
```

EID	EName	DeptID	Salary	PID	ProjectName	EID
101	Arun	1	60000	1	Website	101
103	Kumar	1	70000	2	App	103
105	Siva	2	55000	3	Software	105

3 rows in set (0.00 sec)

```

--v SELECT *
--v FROM Employee
--v RIGHT JOIN Project
--v ON Employee.EID=Project.EID;
+-----+-----+-----+-----+-----+-----+-----+
| EID | EName | DeptID | Salary | PID | ProjectName | EID |
+-----+-----+-----+-----+-----+-----+-----+
| 101 | Arun | 1 | 60000 | 1 | Website | 101 |
| 102 | Bala | 2 | 45000 | NULL | NULL | NULL |
| 103 | Kumar | 1 | 70000 | 2 | App | 103 |
| 104 | Rahul | 3 | 40000 | NULL | NULL | NULL |
| 105 | Siva | 2 | 55000 | 3 | Software | 105 |
+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT *
--v FROM Employee
--v CROSS JOIN Project;
+-----+-----+-----+-----+-----+-----+-----+
| EID | EName | DeptID | Salary | PID | ProjectName | EID |
+-----+-----+-----+-----+-----+-----+-----+
| 101 | Arun | 1 | 60000 | 3 | Software | 105 |
| 101 | Arun | 1 | 60000 | 2 | App | 103 |
| 101 | Arun | 1 | 60000 | 1 | Website | 101 |
| 102 | Bala | 2 | 45000 | 3 | Software | 105 |
| 102 | Bala | 2 | 45000 | 2 | App | 103 |
| 102 | Bala | 2 | 45000 | 1 | Website | 101 |
| 103 | Kumar | 1 | 70000 | 3 | Software | 105 |
| 103 | Kumar | 1 | 70000 | 2 | App | 103 |
| 103 | Kumar | 1 | 70000 | 1 | Website | 101 |
| 104 | Rahul | 3 | 40000 | 3 | Software | 105 |
| 104 | Rahul | 3 | 40000 | 2 | App | 103 |
| 104 | Rahul | 3 | 40000 | 1 | Website | 101 |
| 105 | Siva | 2 | 55000 | 3 | Software | 105 |
| 105 | Siva | 2 | 55000 | 2 | App | 103 |
| 105 | Siva | 2 | 55000 | 1 | Website | 101 |
+-----+-----+-----+-----+-----+-----+-----+
15 rows in set (0.00 sec)

```

- 6) Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.

```

mysql> CREATE VIEW DeptSalaryView AS
--v SELECT
--v Department.DeptName,
--v COUNT(Employee.EID) AS TotalEmployees,
--v AVG(Employee.Salary) AS AverageSalary
--v FROM Employee
--v JOIN Department
--v ON Employee.DeptID=Department.DeptID
--v GROUP BY Department.DeptName;
Query OK, 0 rows affected (0.06 sec)

mysql> SELECT * FROM DeptSalaryView;
+-----+-----+-----+
| DeptName | TotalEmployees | AverageSalary |
+-----+-----+-----+
| IT | 2 | 65000.0000 |
| HR | 2 | 50000.0000 |
| Sales | 1 | 40000.0000 |
+-----+-----+-----+
3 rows in set (0.05 sec)

```

- 7) Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

```
mysql> SELECT * FROM DeptSalaryView;
+-----+-----+-----+
| DeptName | TotalEmployees | AverageSalary |
+-----+-----+-----+
| IT       | 2              | 65000.0000    |
| HR       | 2              | 50000.0000    |
| Sales    | 1              | 40000.0000    |
+-----+-----+-----+
3 rows in set (0.05 sec)

mysql> INSERT INTO STUDENT
-> VALUES
-> (101, 'Arun', '2005-01-15', 1);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM STUDENT;
+-----+-----+-----+-----+
| SID | Name | DOB       | DeptID |
+-----+-----+-----+-----+
| 101 | Arun | 2005-01-15 | 1      |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> UPDATE STUDENT
-> SET Name='Karthik'
-> WHERE SID=101;
Query OK, 1 row affected (0.06 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM STUDENT;
+-----+-----+-----+-----+
| SID | Name   | DOB       | DeptID |
+-----+-----+-----+-----+
| 101 | Karthik | 2005-01-15 | 1      |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> DELETE FROM STUDENT
-> WHERE SID=101;
Query OK, 1 row affected (0.05 sec)

mysql> SELECT * FROM STUDENT;
Empty set (0.00 sec)
```

- 8) Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.

```
mysql> SELECT *
-> FROM Employee
-> INNER JOIN Department
-> ON Employee.DeptID=Department.DeptID
-> WHERE DeptName='IT'
-> AND Salary>50000;
+-----+-----+-----+-----+-----+-----+
| EID | EName | DeptID | Salary | DeptID | DeptName |
+-----+-----+-----+-----+-----+-----+
| 101 | Arun | 1 | 60000 | 1 | IT |
| 103 | Kumar | 1 | 70000 | 1 | IT |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.03 sec)
```

- 9) Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.

```
mysql> SELECT PassengerName,FlightName
-> FROM Passenger
-> JOIN Reservation
-> ON Passenger.PassengerID=Reservation.PassengerID
-> JOIN Flight
-> ON Reservation.FlightID=Flight.FlightID;
+-----+-----+
| PassengerName | FlightName |
+-----+-----+
| Arun          | Indigo     |
| Bala          | Air India  |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT *
-> FROM Passenger
-> LEFT JOIN Reservation
-> ON Passenger.PassengerID=Reservation.PassengerID;
+-----+-----+-----+-----+-----+-----+
| PassengerID | PassengerName | ReservationID | PassengerID | FlightID | JourneyDate |
+-----+-----+-----+-----+-----+-----+
| 1           | Arun          | 1             | 1           | 101      | 2026-08-15  |
| 2           | Bala          | 2             | 2           | 102      | 2026-08-18  |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT PassengerName
-> FROM Passenger
-> WHERE PassengerID IN
-> (
-> SELECT PassengerID
-> FROM Reservation
-> );
+-----+
| PassengerName |
+-----+
| Arun          |
| Bala          |
+-----+
2 rows in set (0.04 sec)
```

```
mysql> CREATE VIEW FlightDetails AS
-> SELECT FlightName,Source,Destination
-> FROM Flight;
Query OK, 0 rows affected (0.05 sec)

mysql> SELECT *
-> FROM FlightDetails;
+-----+-----+-----+
| FlightName | Source  | Destination |
+-----+-----+-----+
| Indigo     | Chennai | Delhi       |
| Air India  | Chennai | Mumbai      |
+-----+-----+-----+
2 rows in set (0.00 sec)
```